vazul: An R Package for Analysis Blinding

Abstract

Analysis blinding is a methodological approach designed to protect the confirmatory status of an analysis by concealing crucial test-relevant aspects of the data from the analysts. This article introduces the vazul package for R, which provides a comprehensive suite of tools for implementing masking and scrambling techniques. We discuss the theoretical foundations of analysis blinding, particularly as a complement or alternative to preregistration.

The package allows researchers to maintain analytic flexibility while safeguarding against bias. By using vazul, data managers can easily generate blinded datasets that preserve essential distributional properties while obscuring associations or labels that could lead to biased decisions. We provide detailed examples of vector-level, data-frame-level, and row-wise blinding operations, and demonstrate how the package handles complex data structures such as hierarchical or repeated-measures designs.

Keywords: analysis blinding, R, open science, reproducible research, masking, scrambling

vazul: An R Package for Analysis Blinding

Introduction

In data analysis, researchers face the challenge of maintaining objectivity and minimizing bias. A central concern is the considerable analytic flexibility present at various stages of the workflow, including data preprocessing, variable selection, model specification, and statistical testing. This flexibility has been described as the "garden of forking paths" (Gelman & Loken, 2013), meaning that possibly hundreds or thousands of plausible, non-redundant analytic strategies can be applied to the same dataset (Simonsohn, Nelson, & Simmons, 2020; Steegen, Tuerlinckx, Gelman, & Vanpaemel, 2016). Crucially, different analytic paths can yield different results, and analysts may, intentionally or not, gravitate toward those that align with their expectations.

The result can be biased or even spurious findings (Gelman & Loken, 2016; Ioannidis, 2005b; Simmons, Nelson, & Simonsohn, 2011), which have eroded trust in science and contributed to replication failures across medical sciences, psychology, economics, and computer science (Baker, 2016; Camerer et al., 2016; Cockburn, Dragicevic, Besançon, & Gutwin, 2020; Ioannidis, 2005a; Nosek et al., 2022; Open Science Collaboration, 2015; Pashler & Wagenmakers, 2012).[1] To ensure the reliability of empirical findings, it is therefore essential to prevent the idiosyncrasies of the data from feeding back into the formulation of the hypotheses being tested (HARKing, Kerr, 1998) or from prompting researchers to exploit their analytic freedom to accentuate desired outcomes (Simmons et al., 2011). In response, researchers have proposed methodologies to limit the influence of bias in data analysis. One such method is analysis blinding, which supplies analysts with an altered version of the real data that conceals biasing features until the analysis plan is finalized.

[1] But see Gilbert, King, Pettigrew, and Wilson (2016), Muradchanian, Hoekstra, Kiers, and Ravenzwaaij (2021), Savitz et al. (2024), and McShane, Bradlow, Lynch Jr, and Meyer (2024) for critiques of how replication failure is measured and of the design and validity of some replication attempts.

Analysis blinding, or blind analysis (R. MacCoun & Perlmutter, 2015, 2018), is a methodological approach designed to protect the confirmatory status of an analysis by concealing crucial test-relevant aspects of the data from the analysts—for example, by scrambling dependent variables or masking condition labels. The procedure typically involves two independent parties: a data manager, who has full access to the raw data and is responsible for implementing the blinding steps, and an analyst, who is tasked with developing the analysis plan. After the data manager applies the blinding procedure, the analyst receives the resulting modified dataset: the blinded data.

Working with the blinded data, the analyst can explore and preprocess the dataset and develop an analysis plan without being influenced by whether a particular analytic choice produces the desired result, as any patterns in the blinded data are, by design, only a product of chance. Importantly, although the test-relevant elements are concealed, other meaningful characteristics remain intact, including demographic information, secondary variables, and the distributional properties of both dependent and independent variables. As a result, the analyst retains all information necessary to tailor the analytic approach to the specific, and potentially unexpected, features of the data. Once the analysis plan has been finalized using the blinded data, the analyst is granted access to the raw, unblinded dataset, and the confirmatory analysis can then be carried out strictly according to the pre-established plan.

Analysis blinding ties in with other methodologies to minimize bias in the research cycle, such as single-blind or double-blind experimental designs, where participants and/or experimenters are unaware of certain aspects of the study to prevent bias in data collection (Schulz & Grimes, 2002). Here, the analysts are intentionally kept unaware of certain aspects of the data to prevent bias in data analysis, which is why the methodology is also referred to as triple-blind (Schulz & Grimes, 2002).

The Two-Stage Workflow

To maintain the integrity of the blinding process, researchers should adhere to a strict two-stage workflow.

Stage 1: Blinded analysis script development. In this stage, the data manager provides the analyst with a version of the data that has been blinded (e.g., processed using **vazul**). Crucially, the blinding transformation must be non-reversible, and the data manager must securely retain the original, unblinded dataset for use in the second stage. The analyst should not be able to reverse-engineer the masks or un-scramble the values to recover the original associations. All data cleaning, visualization, and model building are performed on this blinded set.

Stage 2: Unblinding and execution. Once the analysis code is finalized and locked (e.g., by committing it to a repository or uploading it to a preregistration platform), the second stage begins. During unblinding, the analyst does not modify the code to accommodate the real data. Instead, they apply the finalized analysis code to the original, unblinded dataset. This ensures that the results are the product of decisions made in total ignorance of the statistical inferences (e.g., p-values) or effect sizes.

Although many blinding techniques are conceptually straightforward, such as scrambling outcome variables or masking variable names and factor labels, their practical implementation can become technically challenging once real research designs and data structures are considered. For instance, in hierarchical models, researchers may need to scramble the outcome variable to destroy the effect of interest while still preserving information within grouping levels, such as country-level means in cross-cultural datasets (e.g., Sarafoglou, Hoogeveen, & Wagenmakers, 2023a). Similarly, in repeated-measures designs, condition labels may need to be masked so that each participant's condition structure is preserved while permuting the mapping of conditions across participants to obscure the true experimental effect. These complexities make manual analysis blinding

difficult to carry out consistently and correctly. To lower barriers for data managers and facilitate the broader adoption of analysis blinding in research labs, dedicated software tools, such as **R** packages, are needed to implement these procedures reliably.

To meet this need, the **vazul**[2] package (Nagy, Kovács, & Sarafoglou, 2026) in **R** (R Core Team, 2025) offers flexible and reproducible tools for implementing analysis blinding. The **R** package provides functionality for data scrambling as well as data and variable masking. In the remainder of this article, we introduce the methodology in more detail, describe the functionalities of the **vazul** package, and illustrate its application through practical examples. We conclude with a brief discussion and outline future directions.

Analysis Blinding: Safeguarding Against Bias Without Preregistration

Originally, analysis blinding gained traction in astrophysics in the early 2000s (R. MacCoun & Perlmutter, 2018) as a means of safeguarding analysts against bias. Since then, the methodology has also been advocated in the social and behavioral sciences as an alternative or complement to preregistration (Aczel et al., 2020; Dutilh, Sarafoglou, & Wagenmakers, 2019; Robert MacCoun, 2021; R. MacCoun & Perlmutter, 2015, 2018; Nagy et al., 2025).

This interest partly reflects the fact that analysis blinding addresses practical limitations of preregistration when analytic flexibility is required. While preregistration provides a valuable framework for increasing transparency, analysts may nonetheless encounter unexpected features of the data that necessitate deviations from the preregistered plan. Such deviations are common and often unavoidable, and when transparently reported and justified, they can enhance the credibility and validity of research findings (e.g., Van

[2] Vazul was an 11th-century Hungarian prince of the Árpád dynasty, a cousin of King Stephen I (c. 975–1038). Around 1031, Vazul was blinded—according to contemporary chronicles-to make him unfit for succession to the throne. In the same spirit, the **vazul** package helps create blinded data that are unfit for p-hacking.

Den Akker et al., 2024; Lakens, 2024; Schneider, Rosman, Kelava, & Merk, 2022; Song, Markowitz, & Taylor, 2022).

At the same time, preregistration offers limited guidance on how analytic decisions should be adapted in response to unforeseen data characteristics, and how such deviations should be interpreted with respect to confirmatory status. As a result, researchers may face a tension between adhering strictly to a preregistered plan and making principled, data-informed adjustments. Analysis blinding alleviates this tension by allowing analysts to flexibly develop and refine analysis plans using blinded data, before any test-relevant information is revealed.

In contrast to preregistration, analysis blinding allows researchers to maintain flexibility in their analysis plans, as they can explore the data without being constrained by a rigid preregistered protocol. This flexibility is especially important in analyses involving advanced statistical modeling or preprocessing, where not all decisions can be anticipated in advance and researchers must make data-dependent choices. For instance, many psychological constructs, such as well-being, are measured with multi-item scales whose factor structure must be examined before further analysis; whether the items load onto a single factor or multiple factors determines subsequent analytic steps, including whether composite scores can be computed or whether a more complex measurement model is required (see e.g., Hoogeveen, Sarafoglou, van Elk, & Wagenmakers, 2023). The flexibility gained by analysis blinding enables researchers to develop analysis strategies that are appropriately tailored to the specific characteristics of the dataset without the need to anticipate and specify all eventualities in advance. Consequently, researchers who analyze their data with analysis blinding are less likely to deviate from their developed analysis plan compared to researchers who preregister their analytic strategy in a text-based format (Sarafoglou et al., 2023a).

Moreover, similar in spirit to approaches proposed in code-based preregistration (Peikert, Van Lissa, & Brandmaier, 2021; Van Lissa et al., 2021), analysis blinding naturally

produces analysis plans that are specific, precise, and exhaustive, because they are written directly in code rather than described verbally in preregistration templates. As with preregistration, analysis scripts based on the blinded data can be uploaded or attached to preregistration platforms (e.g., the Open Science Framework) to provide a transparent record of the planned analysis before the data are unblinded. In short, analysis blinding can serve as a valuable alternative or additional safeguard to preregistered analysis plans: it effectively protects against bias while providing analysts with the flexibility needed to develop an analysis strategy that is optimally suited to their data. Notably, even when the blinding code itself is shared, the original data cannot be recovered from the blinded dataset without access to the raw data, making analysis blinding compatible with transparent and open workflows.

Analysis blinding can be particularly valuable for secondary data analysis, where the confirmatory power of preregistration is often compromised by prior exposure to the data (Thibault et al., 2023). In archival datasets, large-scale surveys, or shared repositories, researchers may already possess unavoidable knowledge of variable distributions or previously reported associations. By developing and finalizing analysis plans on a blinded version of the existing data, researchers can maintain analytic flexibility while providing a verifiable safeguard against post-hoc bias. This approach effectively protects the confirmatory status of the study in contexts where traditional preregistration might otherwise be perceived as impractical or insufficient.

Overview of the main functions

The package provides functions for two main types of analysis blinding:

1. **Masking**: Replaces original values with anonymous labels, completely hiding the original information.
2. **Scrambling**: Randomizes the order of existing values while preserving all original data content.

Choosing between masking and scrambling

When choosing between masking and scrambling for analysis blinding, the decision usually turns on what information must be concealed and what structure must be preserved to allow for the development of a suitable analysis pipeline. Masking is most appropriate when the risk lies in the labels of categories rather than their relationships. By replacing group names (e.g., 'Treatment' and 'Control') with arbitrary codes (e.g., 'Group A' and 'Group B'), the group allocation is preserved. This allows analysts to perform group-level operations—such as ANOVA or regression—while remaining blinded to the substantive identity of those groups. Note, that since group-level distinctions remain intact, masking is best suited for workflows where knowing the direction of a difference would not inadvertently reveal the underlying category meaning. Masking also accommodates cases where different variables require different relabeling schemes or prefixes, for example when several independent categorical factors need to remain distinguishable but substantively opaque. This approach is common in clinical trials, behavioral experiments, and preregistered confirmatory analyses where preserving the structure of factors is essential but knowledge of factor identity would bias analytic choices. For instance, an analyst can check for data consistency across four masked study sites (e.g., "Site A" through "Site D") to ensure uniform data quality without knowing the specific geographic locations. This preserves the site-level grouping for variance testing while preventing biased assumptions about data from specific regions.

Masking can be indispensable in situations where the variable names themselves carry substantive meaning. This occurs, for example, in network analysis, where variables become nodes with interpretable labels, or in factor-analytic work, where item names often hint at their latent content (Hoekstra, Huth, Sekulovski, Delhalle, & Sarafoglou, 2025). In such settings, simply scrambling values would not prevent analysts from inferring the underlying constructs, whereas masking removes this semantic cue entirely. The **vazul** package supports

this use case directly by offering functionality for masking variable names alongside their values.

Scrambling becomes the better choice when the goal is to preserve the marginal distributions of numeric variables while severing their hypothesized associations. It is particularly useful in settings where analysts need access to realistic distributions—variance, skew—to make decisions about transformations, model families, or robustness checks, yet must remain blind to the relationship between predictors and outcomes. By permuting values within variables, scrambling prevents recognition of treatment or experimental effects, correlations, or temporal patterns, making it harder to inadvertently tune an analysis toward the desired result. It is often the preferred option in longitudinal studies, high-dimensional observational datasets where categorical relabeling would either destroy important numeric structure or be easy to reverse-engineer (e.g., in case of three experimental conditions, the researcher may guess which one is which based on the means). Scrambling is generally the more versatile option, because it can be applied across variable types and adapts well to a wide range of study designs and analytic workflows. Its ability to break associations while preserving distributions makes it suitable for almost everything from tightly controlled experiments to complex observational datasets, including high-dimensional or mixed-format data (Sarafoglou & Hoogeveen, 2025; Sarafoglou, Hoogeveen, & Wagenmakers, 2023b).

Table 1

Comparison of masking and scrambling approaches for analysis blinding.

Aspect	**Masking**	**Scrambling**
Original values	Replaced with arbitrary codes	Preserved but permuted
Data distribution	Category counts preserved with different labels	Marginal distributions fully preserved

Aspect	Masking	Scrambling
Associations with blinded variables	Preserved	Broken (correlations, treatment–outcome links removed)
Risk addressed	Hiding the identity or meaning of categories or variable names	Preventing recognition of true relationships or effects
Best suited for	Categorical variables with many different labels; variables whose names convey substantive meaning	Numeric or categorical variables; mixed-type data
Typical use cases	Clinical trials, behavioral experiments, network analysis, factor analysis	Longitudinal data, observational datasets, robustness checks
When essential	When variable names or category labels carry substantive information	When analysts need realistic numeric structure but must remain blind to associations
Flexibility across research settings	Limited by categorical/semantic constraints	Broadly applicable to most data types and study designs

In the **vazul** package, each approach is available at three levels:

- **Vector level**: `mask_labels()` and `scramble_values()` - operate on single vectors
- **Data frame level**: `mask_variables()` and `scramble_variables()` - operate on columns in a data frame
- **Row-wise level**: `scramble_variables(..., .byrow = TRUE)` - operates within rows across columns.

The design of these three levels reflects the common ways researchers interact with data in **R**. Vector-level functions are the primary building blocks, designed for simple

pipelines where a researcher might want to blind a single outcome or a specific grouping factor. These functions are particularly useful when working with **dplyr**'s `mutate()` or within custom functions where a specific variable needs to be isolated and transformed without affecting the rest of the environment (Wickham, François, Henry, Müller, & Vaughan, 2025). By providing these atomic operations, **vazul** ensures that the core blinding logic remains accessible even for non-standard data structures.

Moving to the data frame and row-wise levels, the package addresses more complex research designs. Data frame-level functions allow for bulk operations, which are essential for datasets with dozens of variables requiring consistent masking or independent scrambling. The row-wise operations solve a frequent headache in repeated-measures and longitudinal research. In these designs, the "unit" of analysis is often spread across multiple columns (e.g., independent varibles in factorial designs); the row-wise functions ensure that the relationship within a participant's data is handled correctly—either by scrambling values across those specific time points for each person or by applying a consistent mask that hides the identity of the measurement while preserving the participant-level structure. It is important to note that while **vazul** supports row-wise scrambling, it does not provide a row-wise masking function. Masking across different variables within a single observation lacks a clear methodological justification in analysis blinding, as masking is intended to obscure values while preserving their distributional properties (see later for alternative solutions).

All of the functions require a data frame or vector as input and return a modified version with masked or scrambled values. The original data structure, including class attributes and metadata, is carefully preserved to ensure that the blinded data can be passed directly into existing analysis scripts or visualization packages without requiring additional type conversions.

Table 2

*Overview of **vazul** functions for masking and scrambling data*

Function	Level	Purpose
`mask_labels()`	Vector	Replace categorical values with anonymous labels
`mask_variables()`	Data frame	Mask multiple columns
`mask_names()`	Variable names	Mask column names
`scramble_values()`	Vector	Randomize value order
`scramble_variables()`	Data frame	Scramble multiple columns
`scramble_variables(..., .byrow = TRUE)`	Row-wise	Scramble values within rows

The **vazul** package is open source and distributed via CRAN and GitHub (https://github.com/nthun/vazul). All analyses reported here were conducted using version 1.1.0.

In the following sections, we provide detailed descriptions and examples for each of these functions, illustrating their use cases and practical applications in research workflows. We starts with setting up the environment and loading the included datasets.

Included datasets

The **vazul** package includes two research datasets for demonstration and practice. The `marp` dataset contains cross-national survey data on religiosity (Hoogeveen et al., 2023), while the `williams` dataset contains experimental data from a stereotyping study (Holzmeister et al., 2025; Sarafoglou & Hoogeveen, 2025).

Masking functions

Masking functions replace categorical values with anonymous labels. This is useful when you want to hide the original information, such as treatment conditions or group assignments, and there are a limited number of unique values. The `mask_labels()` function takes a character or factor type vector and replaces each unique value with a randomly assigned masked label. The function preserves factor structure when the input is a factor. After masking, each unique value receives a unique masked label, the same original value always maps to the same masked label, and the assignment of masked labels to original values is randomized. Missing values are preserved during masking.

```
# Create a simple treatment vector
treatment <- c("control", "treatment_1", "treatment_2", "control", "treatment_1", "treat

# Mask the labels
mask_labels(treatment)
```

```
## [1] "masked_group_01" "masked_group_02" "masked_group_03" "masked_group_01"
## [5] "masked_group_02" "masked_group_03"
```

It is possible to customize the prefix used for masked labels:

```
mask_labels(treatment, prefix = "group_")
```

```
## [1] "group_01" "group_02" "group_03" "group_01" "group_02" "group_03"
```

The `mask_variables()` function extends the masking functionality to multiple columns in a data frame simultaneously. It is possible to use tidyelect helpers to select columns. for instance, in the MARP dataset, we want to simultaneously mask labels within

the columns "country" and "denomination". To make the example manageable, we first create a smaller example dataset with 10 random rows. Then we demonstrate how to use `mask_variables()` to mask the selected columns. It is possible to use tidyselect helpers (such as `starts_with()`, `any_of()`, etc.) to select columns. In the next example, we apply masking to all character columns in the example dataset.

```
marp_example <-
    marp |>
    select(subject, country, denomination) |>
    sample_n(6)


marp_example
```

```
##   subject country               denomination
## 1    5011   India Christian (Roman Catholic)
## 2    4015 Germany Christian (Roman Catholic)
## 3    4271 Germany                Evangelical
## 4    4591 Germany                       <NA>
## 5   10225      US Christian (Roman Catholic)
## 6    9060   Spain                       <NA>
```

```
marp_example |>
    mask_variables(c("country", "denomination"))
```

```
##   subject          country          denomination
## 1    5011 country_group_01 denomination_group_01
## 2    4015 country_group_03 denomination_group_01
## 3    4271 country_group_03 denomination_group_02
```

```
## 4    4591 country_group_03                   <NA>
## 5   10225 country_group_04 denomination_group_01
## 6    9060 country_group_02                   <NA>
```

```
marp_example |>
    mask_variables(where(is.character))
```

```
##   subject          country          denomination
## 1    5011 country_group_04 denomination_group_01
## 2    4015 country_group_02 denomination_group_01
## 3    4271 country_group_02 denomination_group_02
## 4    4591 country_group_02                   <NA>
## 5   10225 country_group_03 denomination_group_01
## 6    9060 country_group_01                   <NA>
```

By default, each column gets its own set of masked labels with the column name as prefix. When `.across_variables = TRUE`, all selected columns share the same mapping. A crucial feature for longitudinal or repeated-measures data in wide format is the `.across_variables` parameter. When set to `TRUE`, it ensures that all selected variables are masked using the same transformation or mapping. This preserves the relative differences or associtions between time points while hiding the true values from the analyst. For example, in the dataset below, we have two columns representing pre- and post-treatment conditions. By applying `mask_variables()` with `.across_variables = TRUE`, we ensure that the same original condition (e.g., “A”) is masked to the same new label across both columns, preserving the relationship between pre- and post-conditions while concealing their true identities.

```r
df <- data.frame(pre_condition = c("A", "B", "C", "D"),
                 post_condition = c("B", "D", "D", "C"),
                 id = c(1, 2, 3, 4)
                 )
df
```

```
##   pre_condition post_condition id
## 1             A              B  1
## 2             B              D  2
## 3             C              D  3
## 4             D              C  4
```

```r
mask_variables(df,
               c("pre_condition", "post_condition"),
               .across_variables = TRUE)
```

```
##     pre_condition  post_condition id
## 1 masked_group_02 masked_group_01  1
## 2 masked_group_01 masked_group_04  2
## 3 masked_group_03 masked_group_04  3
## 4 masked_group_04 masked_group_03  4
```

Masking variable names. The `mask_names()` function allows users to rename variables in a dataset to anonymous, generic labels. This is especially useful in analyses where variable names carry substantive meaning (e.g., questionnaire items, network nodes, test-items), and one wants to prevent analysts from recognizing which item is which during preprocessing or exploratory phases.

The `mask_names()` function takes a data frame and one or more variable sets, which can be specified either with tidyselect expressions (e.g., `starts_with("Q")`) or as character vectors of column names. The `prefix` argument sets the base for the masked names. For instance, in the `williams` data we can blind all variables that are related to the impulsivity scale.

```
names(williams)
```

```
##  [1] "subject"              "SexUnres_1"           "SexUnres_2"
##  [4] "SexUnres_3"           "SexUnres_4_r"         "SexUnres_5_r"
##  [7] "Impuls_1"             "Impuls_2_r"           "Impul_3_r"
## [10] "Opport_1"             "Opport_2"             "Opport_3"
## [13] "Opport_4"             "Opport_5"             "Opport_6_r"
## [16] "InvEdu_1_r"           "InvEdu_2_r"           "InvChild_1"
## [19] "InvChild_2_r"         "age"                  "gender"
## [22] "ecology"              "duration_in_seconds"  "attention_1"
## [25] "attention_2"
```

```
williams |>
    mask_names(starts_with("Impul"), prefix = "masked_") |>
    names()
```

```
##  [1] "subject"              "SexUnres_1"           "SexUnres_2"
##  [4] "SexUnres_3"           "SexUnres_4_r"         "SexUnres_5_r"
##  [7] "masked_02"            "masked_01"            "masked_03"
## [10] "Opport_1"             "Opport_2"             "Opport_3"
## [13] "Opport_4"             "Opport_5"             "Opport_6_r"
## [16] "InvEdu_1_r"           "InvEdu_2_r"           "InvChild_1"
```

```
## [19] "InvChild_2_r"        "age"                  "gender"
## [22] "ecology"             "duration_in_seconds" "attention_1"
## [25] "attention_2"
```

Use multiple `mask_names()` calls to blind different sets of variables independently. Assigning distinct prefixes allows variables to be anonymized while preserving information about their original clusters. For example, an analyst can recognize that a set of variables belongs to the same construct—enabling procedures such as factor analysis or reliability estimation—without knowing which specific items are being evaluated. The example below demonstrates this clustered masking approach in a pipe-based workflow.

```
masked_williams <-
    williams |>
    mask_names(starts_with("SexUnres"), prefix = "masked_set_A_") |>
    mask_names(starts_with("Impul"), prefix = "masked_set_B_") |>
    mask_names(starts_with("Opport"), prefix = "masked_set_C_") |>
    mask_names(starts_with("InvEdu"), prefix = "masked_set_D_") |>
    mask_names(starts_with("InvChild"), prefix = "masked_set_E_")

names(masked_williams)
```

```
##  [1] "subject"            "masked_set_A_03"    "masked_set_A_04"
##  [4] "masked_set_A_01"    "masked_set_A_02"    "masked_set_A_05"
##  [7] "masked_set_B_01"    "masked_set_B_02"    "masked_set_B_03"
## [10] "masked_set_C_06"    "masked_set_C_01"    "masked_set_C_05"
## [13] "masked_set_C_02"    "masked_set_C_04"    "masked_set_C_03"
## [16] "masked_set_D_01"    "masked_set_D_02"    "masked_set_E_02"
## [19] "masked_set_E_01"    "age"                "gender"
```

```
## [22] "ecology"             "duration_in_seconds" "attention_1"
## [25] "attention_2"
```

Once masked, one can proceed with analyses (e.g. factor analysis, network analysis) on the masked variables, without knowing which original items correspond to which columns. For example, we can run a factor analysis on the masked items:

```
masked_williams |>
  select(starts_with("masked_set")) |>
  factanal(factors = 3, rotation = "varimax") |>
  loadings() |>
  print(cutoff = 0.3, sort = TRUE)
```

```
##
## Loadings:
##                 Factor1 Factor2 Factor3
## masked_set_A_03  0.648           0.504
## masked_set_A_04  0.705
## masked_set_A_01  0.730           0.340
## masked_set_B_01  0.751
## masked_set_C_06  0.873
## masked_set_C_01  0.833
## masked_set_C_05  0.866
## masked_set_C_02  0.761
## masked_set_C_04  0.863
## masked_set_E_02  0.770
## masked_set_A_02          0.745   0.306
## masked_set_A_05          0.568
```

```
## masked_set_B_02          0.861
## masked_set_B_03          0.740
## masked_set_C_03          0.696
## masked_set_D_01          0.695
## masked_set_D_02          0.841
## masked_set_E_01          0.865
##
##                Factor1 Factor2 Factor3
## SS loadings      6.314   4.842   0.731
## Proportion Var   0.351   0.269   0.041
## Cumulative Var   0.351   0.620   0.660
```

Because the column names are anonymized, any decisions about factor inclusion or rotation are made blind to the substantive meaning of items, reducing the risk of interpretive bias.

In short, `mask_names()` supports naming-based blinding by decoupling analysis from semantic content, while preserving the full data structure needed for legitimate exploratory workflows.

Scrambling functions

Scrambling functions randomize the order of values while preserving all original data content. This approach maintains the data distribution while breaking the connection between observations and their original values. The `scramble_values()` function randomly reorders the elements of a vector. The vector can contain numeric, character, or factor type data. Scrambling is useful when we want to preserve the original values but eliminate any correspondence between observations and their values.

```r
# Numeric data
numbers <- 1:10
scramble_values(numbers)
```

```
##  [1]  2  1  5  7  9 10  3  8  6  4
```

The `scramble_variables()` function extends the scrambling functionality to data frames. It allows scrambling multiple columns simultaneously, with options for independent scrambling, joint scrambling, and within-group scrambling. Columns can be selected using tidyselect helpers. For instance, in the williams data we could choose to scramble the columns age (a demographic variable) and ecology (the identifier of the experimental condition) as follows:

```r
williams_example <-
    williams |>
    select(subject, age, ecology) |>
    head()

williams_example
```

```
## # A tibble: 6 x 3
##   subject          age ecology
##   <chr>          <dbl> <chr>
## 1 A30MP4LXV4MIFD    34 Hopeful
## 2 A16X5FB3HAFCKN    30 Desperate
## 3 A1E9D1OT9VJYDZ    40 Desperate
## 4 A16FPOYD7566WI    35 Hopeful
## 5 A11NOTVHWST7Y3    26 Desperate
## 6 A3TDR6MXS6UO5Z    33 Desperate
```

```
scramble_variables(williams_example, c("age", "ecology"))
```

```
## # A tibble: 6 x 3
##   subject          age ecology
##   <chr>          <dbl> <chr>
## 1 A3OMP4LXV4MIFD    30 Desperate
## 2 A16X5FB3HAFCKN    40 Desperate
## 3 A1E9D1OT9VJYDZ    33 Hopeful
## 4 A16FPOYD7566WI    26 Desperate
## 5 A11NOTVHWST7Y3    35 Hopeful
## 6 A3TDR6MXS6UO5Z    34 Desperate
```

By default, columns are scrambled independently. Setting `.together = TRUE` causes the selected columns to be scrambled jointly, preserving their row-wise associations. Adding this argument to the previous example therefore maintains the relationship between the two scrambled variables.

```
scramble_variables(williams_example, c("age", "ecology"), .together = TRUE)
```

```
## # A tibble: 6 x 3
##   subject          age ecology
##   <chr>          <dbl> <chr>
## 1 A3OMP4LXV4MIFD    34 Hopeful
## 2 A16X5FB3HAFCKN    40 Desperate
## 3 A1E9D1OT9VJYDZ    30 Desperate
## 4 A16FPOYD7566WI    35 Hopeful
## 5 A11NOTVHWST7Y3    26 Desperate
## 6 A3TDR6MXS6UO5Z    33 Desperate
```

Scrambling can be done in groups using the `.groups` parameter. This ensures that values are only scrambled within their original groups. In the following example, we scramble the columns x and y within each group defined by the group column:

```
df <- data.frame(x = 1:6,
                 y = letters[1:6],
                 group = c("A", "A", "A", "B", "B", "B")
                 )
df
```

```
##   x y group
## 1 1 a     A
## 2 2 b     A
## 3 3 c     A
## 4 4 d     B
## 5 5 e     B
## 6 6 f     B
```

```
scramble_variables(df, c("x", "y"), .groups = "group")
```

```
## # A tibble: 6 x 3
##       x y     group
##   <int> <chr> <chr>
## 1     2 b     A
## 2     3 c     A
## 3     1 a     A
## 4     5 f     B
## 5     6 e     B
## 6     4 d     B
```

Unlike masking, row-wise operation is highly relevant for scrambling. Row-wise scrambling is performed using `scramble_variables()` with the `.byrow = TRUE` argument. This is useful for scrambling repeated measures or item responses. Within each row, the values are shuffled among the item columns. You can scramble multiple sets of columns independently. The function can use tidyselect helpers. In the next example, we scramble values across three day variables for each participant in a hypothetical dataset:

```
df_wide <- data.frame(
  day_1 = c(1, 4, 7),
  day_2 = c(2, 5, 8),
  day_3 = c(3, 6, 9),
  score_a = c(10, 40, 70),
  score_b = c(20, 50, 80),
  id = 1:3
)


df_wide
```

```
##   day_1 day_2 day_3 score_a score_b id
## 1     1     2     3      10      20  1
## 2     4     5     6      40      50  2
## 3     7     8     9      70      80  3
```

```
scramble_variables(df_wide, starts_with("day_"), c("score_a", "score_b"), .byrow = TRUE)
```

```
##   day_1 day_2 day_3 score_a score_b id
## 1     2    20     3       1      10  1
## 2     4    50     6      40       5  2
## 3     7     8    80      70       9  3
```

Summary

The introduction of **vazul** fills a notable gap in the **R** ecosystem, as no dedicated package currently exists to specifically address the diverse requirements of analysis blinding. While some basic blinding tasks—such as simple vector permutations—can be achieved using base **R** or **tidyverse** functions like sample() or mutate(), these manual approaches quickly become error-prone as study designs grow in complexity. When dealing with multiple independent variables, hierarchical data structures, or the need for consistent masking across several columns, the risk of "breaking" the data structure or accidentally unblinding the analyst increases significantly. By providing a unified and tested framework, **vazul** abstracts these technical intricacies, and allow researchers to focus on the conceptual rigor of their analysis plans rather than the underlying data manipulation logic.

The functions implemented in **vazul** are applicable to a wide range of paradigms and data structures and simplify the blinding process, which in turn facilitates the assignment of data-blinding roles within research teams. Specifically, since data blinding no longer requires implementing custom code the data-manager role need not be filled by an expert in the analytic method or data domain. Instead, junior lab members or external collaborators can manage data blinding, while domain experts develop the analytic strategy.

Beyond standalone use in **R** scripts, the package's modular design makes it an ideal candidate for integration into other statistical software platforms. For instance, the inclusion of **vazul** as a back-end for the **JASP** (JASP Team, 2025) graphical interface could provide a point-and-click solution for analysis blinding, further facilitating the adoption of these best practices across the social and behavioral sciences.

Statements

Open science statement

All code to reproduce this manuscript can be found at OSF https://osf.io/f8mub/overview?view_only=c3548dd0c1a74508acc71beb8a77ec9a

Ethics statement

This study did not involve testing of human participants.

Conflict of interest statement

The authors state no conflicts of interest.

Acknowledgements

XXX was supported by the University Excellence Fund of XXX, and the XXX research fellowship of the XXX. XXX was supported by an XXX.

References

Aczel, B., Szaszi, B., Sarafoglou, A., Kekecs, Z., Kucharský, Š., Benjamin, D., ... Wagenmakers, E.-J. (2020). A consensus-based transparency checklist. *Nature Human Behaviour*, *4*, 4–6. https://doi.org/10.1038/s41562-019-0772-6

Baker, M. (2016). 1,500 scientists lift the lid on reproducibility. *Nature News*, *533*, 452. https://doi.org/10.1038/533452a

Camerer, C. F., Dreber, A., Forsell, E., Ho, T.-H., Huber, J., Johannesson, M., et al.others. (2016). Evaluating replicability of laboratory experiments in economics. *Science*, *351*(6280), 1433–1436. https://doi.org/10.1126/science.aaf0918

Cockburn, A., Dragicevic, P., Besançon, L., & Gutwin, C. (2020). Threats of a replication crisis in empirical computer science. *Communications of the ACM*, *63*(8), 70–79. https://doi.org/10.1145/3360311

Dutilh, G., Sarafoglou, A., & Wagenmakers, E.-J. (2019). Flexible yet fair: Blinding analyses in experimental psychology. *Synthese*, *198*, S5745–S5772. https://doi.org/10.1007/s11229-019-02456-7

Gelman, A., & Loken, E. (2013). The garden of forking paths: Why multiple comparisons can be a problem, even when there is no "fishing expedition" or "p-hacking" and the research hypothesis was posited ahead of time. *Department of Statistics, Columbia University*, *348*. Retrieved from http://www.stat.columbia.edu/~gelman/research/unpublished/p_hacking.pdf

Gelman, A., & Loken, E. (2016). The statistical crisis in science. In M. Pitici (Ed.), *The best writing on mathematics* (Vol. 102, pp. 305–318). Princeton University Press.

Gilbert, D. T., King, G., Pettigrew, S., & Wilson, T. D. (2016). Comment on "estimating the reproducibility of psychological science." *Science*, *351*(6277), 1037–1037. https://doi.org/10.1126/science.aad7243

Hoekstra, R. H. A., Huth, K., Sekulovski, N., Delhalle, M., & Sarafoglou, A. (2025). Safeguarding against bias without preregistration: A tutorial on analysis blinding for network analysis. *PsyArXiv*. https://doi.org/10.31234/osf.io/gruw5/_v1

Holzmeister, F., Johannesson, M., Camerer, C. F., Chen, Y., Ho, T.-H., Hoogeveen, S., ... Dreber, A. (2025). Examining the replicability of online experiments selected by a decision market. *Nature Human Behaviour*, *9*(2), 316–330. https://doi.org/10.1038/s41562-024-02062-9

Hoogeveen, S., Sarafoglou, A., van Elk, M., & Wagenmakers, E.-J. (2023). A many-analysts approach to the relation between religiosity and well-being: Reflection and conclusion. *Religion, Brain, & Behaviour*, *13*(3), 356–363. https://doi.org/10.1080/2153599X.2022.2070263

Ioannidis, J. P. (2005a). Contradicted and initially stronger effects in highly cited clinical research. *JAMA*, *294*(2), 218–228. https://doi.org/10.1001/jama.294.2.218

Ioannidis, J. P. (2005b). Why most published research findings are false. *PLoS Medicine*, *2*(8), e124. https://doi.org/10.1371/journal.pmed.1004085

JASP Team. (2025). *JASP (Version 0.95.4)[Computer software]*. Retrieved from https://jasp-stats.org/

Kerr, N. L. (1998). HARKing: Hypothesizing after the results are known. *Personality and Social Psychology Review*, *2*, 196–217. https://doi.org/10.1207/s15327957pspr0203_4

Lakens, D. (2024). When and how to deviate from a preregistration. *Collabra. Psychology*, *10*(1), 117094. https://doi.org/10.1525/collabra.117094

MacCoun, Robert. (2021). Blinding to remove biases in science and society. In R. Hertwig & C. Engel (Eds.), *Deliberate ignorance: Choosing not to know* (pp. 51–64). Cambridge: MIT Press.

MacCoun, R., & Perlmutter, S. (2015). Blind analysis: Hide results to seek the truth. *Nature*, *526*, 187–190. https://doi.org/10.1038/526187a

MacCoun, R., & Perlmutter, S. (2018). Blind analysis as a correction for confirmatory bias in physics and in psychology. In S. O. Lilienfeld & I. Waldman (Eds.), *Psychological science under scrutiny: Recent challenges and proposed solutions* (pp. 297–322). John Wiley; Sons. https://doi.org/10.1002/9781119095910.ch15

McShane, B. B., Bradlow, E. T., Lynch Jr, J. G., & Meyer, R. J. (2024). "Statistical significance" and statistical reporting: Moving beyond binary. *Journal of Marketing*, 00222429231216910. https://doi.org/10.1177/00222429231216910

Muradchanian, J., Hoekstra, R., Kiers, H., & Ravenzwaaij, D. van. (2021). How best to quantify replication success? A simulation study on the comparison of replication success metrics. *Royal Society Open Science*, *8*, 201697. https://doi.org/10.1098/rsos.201697

Nagy, T., Hergert, J., Elsherif, M. M., Wallrich, L., Schmidt, K., Waltzer, T., … Rubínová, E. (2025). Bestiary of Questionable Research Practices in psychology. *Advances in*

Methods and Practices in Psychological Science, *8*(3), 25152459251348431. https://doi.org/10.1177/25152459251348431

Nagy, T., Kovács, M., & Sarafoglou, A. (2026). *Vazul: An R package for analysis blinding*. Zenodo. https://doi.org/10.5281/ZENODO.18269712

Nosek, B. A., Hardwicke, T. E., Moshontz, H., Allard, A., Corker, K. S., Dreber, A., et al.others. (2022). Replicability, robustness, and reproducibility in psychological science. *Annual Review of Psychology*, *73*, 719–748. https://doi.org/10.1146/annurev-psych-020821-114157

Open Science Collaboration. (2015). Estimating the reproducibility of psychological science. *Science*, *349*, aac4716. https://doi.org/10.1126/science.aac4716

Pashler, H., & Wagenmakers, E.-J. (2012). Editors' introduction to the special section on replicability in psychological science: A crisis of confidence? *Perspectives on Psychological Science*, *7*, 528–530. https://doi.org/10.1177/1745691612465253

Peikert, A., Van Lissa, C. J., & Brandmaier, A. M. (2021). Reproducible research in R: A tutorial on how to do the same thing more than once. *Psych*, *3*(4), 836–867. https://doi.org/10.3390/psych3040053

R Core Team. (2025). *R: A language and environment for statistical computing*. Vienna, Austria: R Foundation for Statistical Computing. Retrieved from https://www.R-project.org/

Sarafoglou, A., & Hoogeveen, S. (2025). Analysis blinding as a potential means to foster a productive collaboration between original authors and replicators. *Collabra. Psychology*, *11*(1), 136869. https://doi.org/10.1525/collabra.136869

Sarafoglou, A., Hoogeveen, S., & Wagenmakers, E.-J. (2023a). Comparing analysis blinding with preregistration in the Many-Analysts Religion Project. *Advances in Methods and Practices in Psychological Science*, *6*(1), 25152459221128319. https://doi.org/10.1177/25152459221128319

Sarafoglou, A., Hoogeveen, S., & Wagenmakers, E.-J. (2023b). Comparing analysis blinding

with preregistration in the many-analysts religion project. *Advances in Methods and Practices in Psychological Science*, *6*(1), 25152459221128319. https://doi.org/10.1177/25152459221128319

Savitz, D. A., Wise, L. A., Bond, J. C., Hatch, E. E., Ncube, C. N., Wesselink, A. K., ... Rothman, K. J. (2024). Responding to reviewers and editors about statistical significance testing. *Annals of Internal Medicine*, *177*, 385–386. https://doi.org/10.7326/M23-2430

Schneider, J., Rosman, T., Kelava, A., & Merk, S. (2022). Do open-science badges increase trust in scientists among undergraduates, scientists, and the public? *Psychological Science*, *33*(9), 1588–1604. https://doi.org/10.1177/09567976221097499

Schulz, K. F., & Grimes, D. A. (2002). Blinding in randomised trials: Hiding who got what. *The Lancet*, *359*(9307), 696–700.

Simmons, J. P., Nelson, L. N., & Simonsohn, U. (2011). False–positive psychology: Undisclosed flexibility in data collection and analysis allows presenting anything as significant. *Psychological Science*, *22*, 1359–1366. https://doi.org/10.1177/0956797611417632

Simonsohn, U., Nelson, L. D., & Simmons, J. P. (2020). Specification curve analysis. *Nature Human Behaviour*, *4*, 1208–1214. https://doi.org/10.1038/s41562-020-0912-z

Song, H., Markowitz, D. M., & Taylor, S. H. (2022). Trusting on the shoulders of open giants? Open science increases trust in science for the public and academics. *The Journal of Communication*, *72*(4), 497–510. https://doi.org/10.1093/joc/jqac017

Steegen, S., Tuerlinckx, F., Gelman, A., & Vanpaemel, W. (2016). Increasing transparency through a multiverse analysis. *Perspectives on Psychological Science*, *11*, 702–712. https://doi.org/10.1177/1745691616658637

Thibault, R. T., Kovacs, M., Hardwicke, T. E., Sarafoglou, A., Ioannidis, J. P. A., & Munafò, M. R. (2023). Reducing bias in secondary data analysis via an explore and confirm analysis workflow (ECAW): A proposal and survey of observational researchers. *Royal Society Open Science*, *10*(10), 230568. https://doi.org/10.1098/rsos.230568

Van Den Akker, O. R., Bakker, M., Van Assen, M. A. L. M., Pennington, C. R., Verweij, L., Elsherif, M. M., ... Wicherts, J. M. (2024). The potential of preregistration in psychology: Assessing preregistration producibility and preregistration-study consistency. *Psychological Methods*. https://doi.org/10.1037/met0000687

Van Lissa, C. J., Brandmaier, A. M., Brinkman, L., Lamprecht, A.-L., Peikert, A., Struiksma, M. E., & Vreede, B. M. (2021). WORCS: A workflow for open reproducible code in science. *Data Science*, *4*(1), 29–49. https://doi.org/10.3233/DS-210031

Wickham, H., François, R., Henry, L., Müller, K., & Vaughan, D. (2025). *Dplyr: A grammar of data manipulation*. Retrieved from https://dplyr.tidyverse.org